

集 成 学 习



TRANSWARP
星 环 科 技



目录 > CONTENTS

- 1 Bagging 和随机森林
- 1 Boosting 和 GBDT
- 2 Stacking

The background features a complex network diagram with numerous nodes (small circles) and connecting lines (edges) forming a web-like structure. The nodes are distributed across the entire frame, with some clusters being denser than others. The lines are thin and grey, creating a subtle, technical aesthetic.

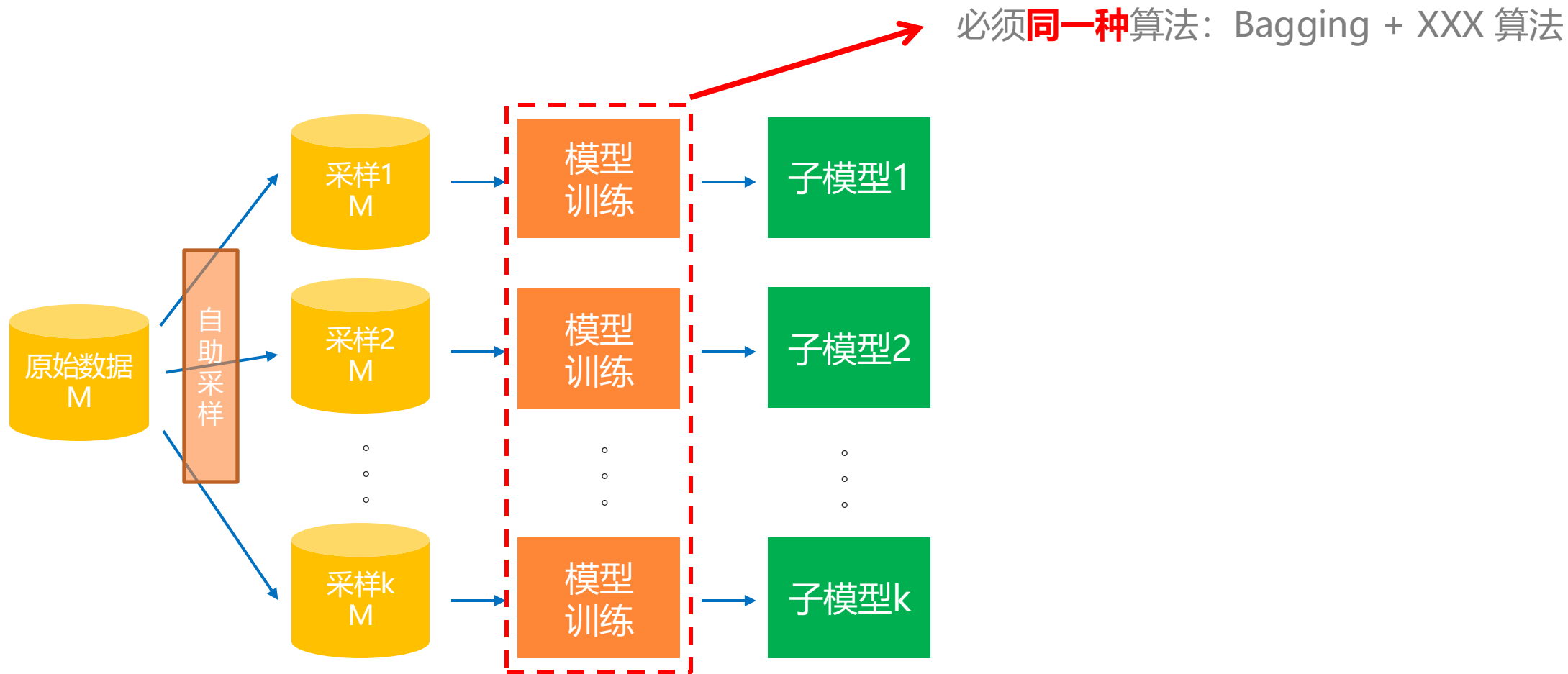
1 chapter

Bagging 和随机森林

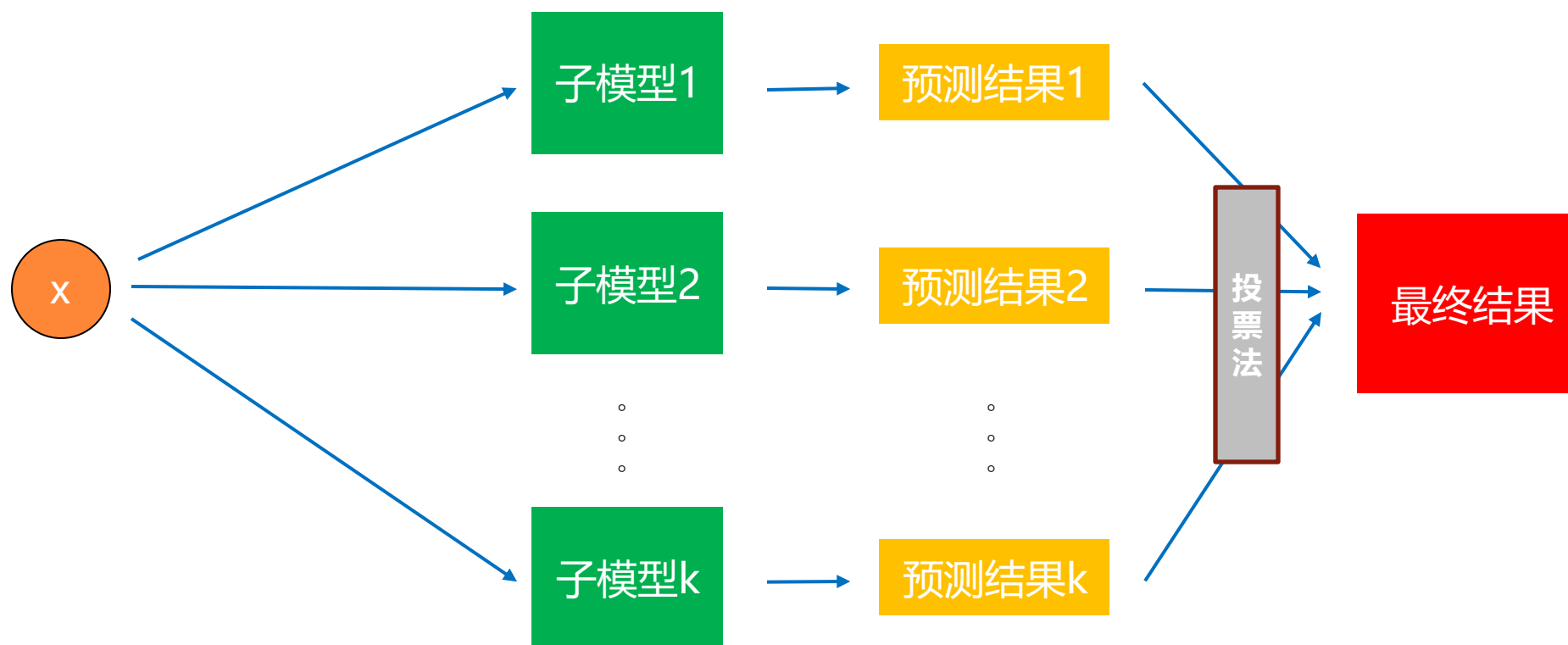
Bagging 融合框架

- Bagging 融合就是采用**有放回抽样**（Bootstrap抽样，又叫重采样，**自助抽样法**）的方法，用**K 次抽样**的样本来训练得到**K 个子模型**，然后对**K 个模型结果**最后进行**Voting**或者**Averaging** 融合。
- 它的主要流程是：
 - 重复进行**K 次** Bootstrap 抽样（每次抽样的样本数量 $M \leq$ 总体样本数量 N ）
 - 对 K 个抽样得到的训练样本进行训练得到K个子模型
 - 对 K 个子模型的结果进行融合：分类用 **voting**，回归用 **Averaging**
- 基于 **Bagging** 融合框架的典型算法代表是**随机森林**，它的子模型采用的是**决策树**算法。

Bagging 融合框架训练流程

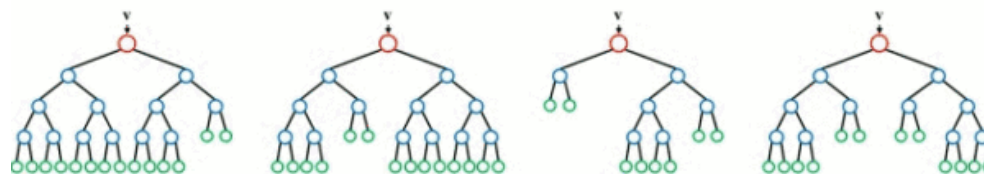


Bagging 融合框架预测流程



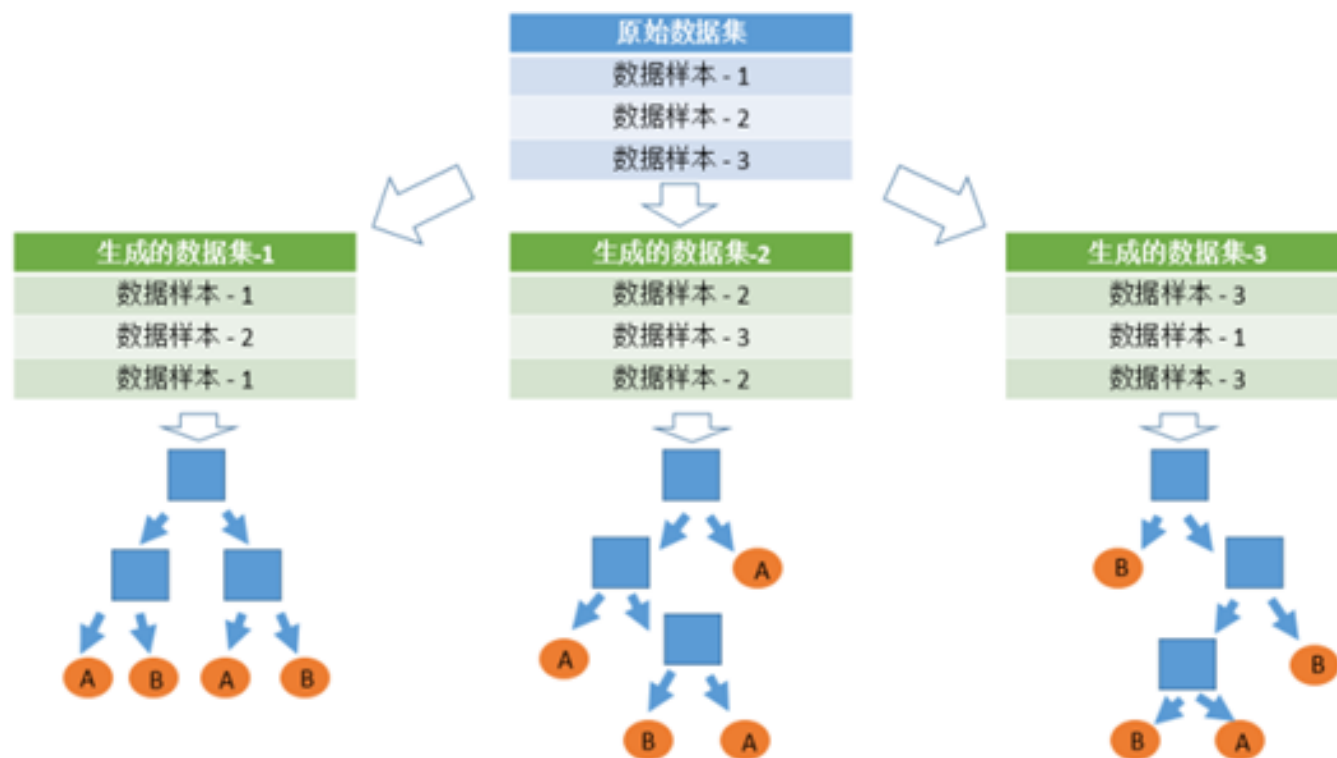
随机森林算法概述

- **随机森林**，顾名思义，就是由**很多决策树**模型**融合**在一起的算法。
- 随机森林是一种**基于Bagging**框架的模型融合算法
- 随机森林的“**森林**”指的就是它的弱模型是由**决策树**算法训练的（具体是CART算法），CART算法既能做分类也能做回归
- 随机森林的“**随机**”指的是构造的若干个弱模型是有一定的“**随机性（两个随机）**”的



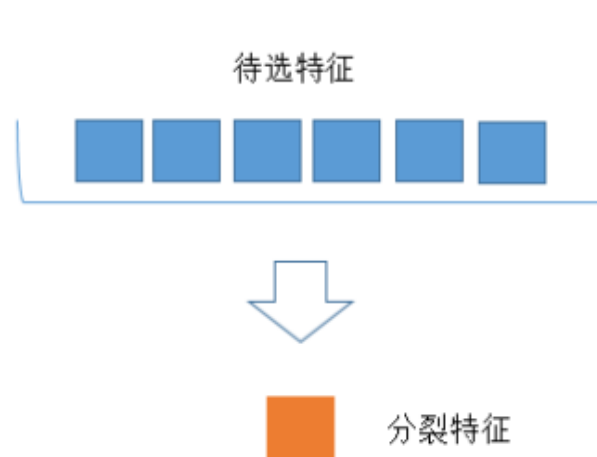
随机森林-两个随机

- **样本随机（行随机）**：对样本进行**有放回随机抽样（bootstrap抽样）**，制造**数据差异**，提升模型**多样性**。

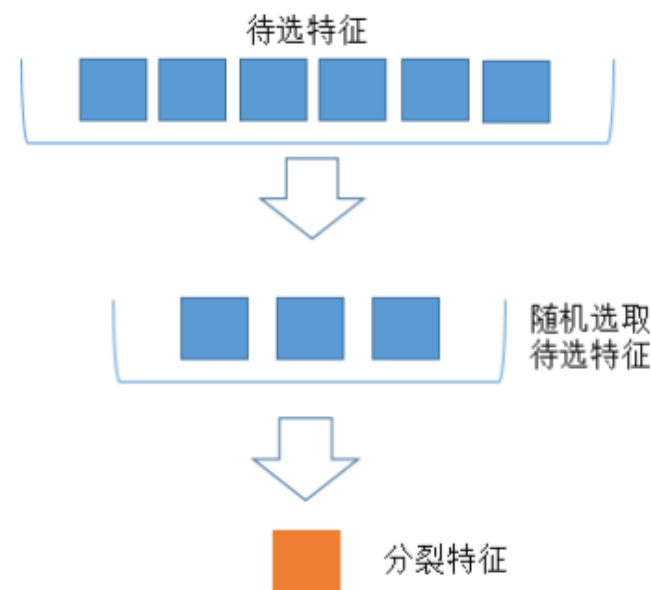


随机森林-两个随机

- **特征随机（列随机）**：子树并未**用到所有**的待选特征，而是从所有特征中随机**选取部分特征**，再在随机选取的特征中选取最优的特征。这样使得树都能够**彼此不同**，提升**多样性**，从而**减小方差**。

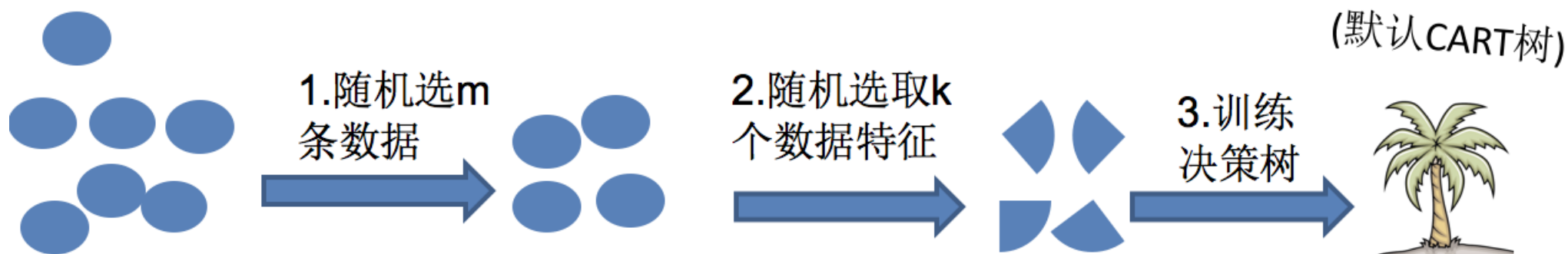


决策树选取分裂特征过程



随机森林子树选取分裂特征过程

随机森林算法流程



4. 重复1-3步构造n个弱决策树

这里是**并行**执行的，并不是循环重复

5. 平权投票集成n个弱决策树

The out-of-bag(oob) error estimate oob误差估计

- **袋外样本**：那些**没有被子模型抽中**的样本就是这个子模型的袋外样本,可以把这些袋外样本作为该子模型的测试集。
- **袋外误差 (out-of-bag error)** 是Bagging融合框架独有的一个术语。随机森林算法由于存在袋外样本，所以，那些**袋外样本**天然的就可以作为测试集对模型进行评估。

随机森林算法示例：

作为示例，我们简化一下算法，假设：

- 我们构建**5个**子模型（决策树）
- 每个子模型**只用1个特征**变量来构建
- 总共有5个特征变量，那么正好每一个子模型用一个特征变量

随机森林算法示例：

CART 1 : Variable Age

	Salary Band	1	2	3
Age	Below 18	90%	10%	0%
	19-27	85%	14%	1%
	28-40	70%	23%	7%
	40-55	60%	35%	5%
	More than 55	70%	25%	5%

CART 2 : Variable Gender

	Salary Band	1	2	3
Gender	Male	70%	27%	3%
	Female	75%	24%	1%

CART 3 : Variable Education

	Salary Band	1	2	3
Education	<=High School	85%	10%	5%
	Diploma	80%	14%	6%
	Bachelors	77%	23%	0%
	Master	62%	35%	3%

CART 4 : Variable Residence

	Salary Band	1	2	3
Residence	Metro	70%	20%	10%
	Non-Metro	65%	20%	15%

CART 5 : Variable Industry

	Salary Band	1	2	3
Industry	Finance	65%	30%	5%
	Manufacturing	60%	35%	5%
	Others	75%	20%	5%

随机森林算法示例：

- 现在给出一个待预测样本：
 - Age : 35 years ;
 - Gender : Male ;
 - Highest Educational Qualification : Diploma holder;
 - Industry : Manufacturing;
 - Residence : Metro.
- 按照Bagging模型融合思想，随机森林模型会这样给出如下的预测结果：把五个子模型给出的**概率值求均值**

CART	Band	1	2	3
Age	28-40	70%	23%	7%
Gender	Male	70%	27%	3%
Education	Diploma	80%	14%	6%
Industry	Manufacturing	60%	35%	5%
Residence	Metro	70%	20%	10%
Final probability		70%	24%	6%

随机森林算法示例：

- 现在给出一个待预测样本：
 - Age : 35 years ;
 - Gender : Male ;
 - Highest Educational Qualification : Diploma holder;
 - Industry : Manufacturing;
 - Residence : Metro.
- 按照Bagging模型融合思想，随机森林模型会这样给出如下的预测结果：把五个子模型给出的**概率值求均值**

CART	Band	1	2	3
Age	28-40	70%	23%	7%
Gender	Male	70%	27%	3%
Education	Diploma	80%	14%	6%
Industry	Manufacturing	60%	35%	5%
Residence	Metro	70%	20%	10%
Final probability		70%	24%	6%

导入工具包

```
import pandas as pd
import numpy as np
from sklearn.feature_extraction import DictVectorizer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV
```

读取数据

```
data=pd.read_csv("train.csv")
data.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

#选择x和y

```
x=data[['Pclass','Age','Sex']]
y=data['Survived']
x.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 3 columns):
Pclass    891 non-null int64
Age       714 non-null float64
Sex       891 non-null object
dtypes: float64(1), int64(1), object(1)
memory usage: 21.0+ KB
```

数据处理

#填充缺失值

```
x = x.fillna(x.mean())  
x.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 891 entries, 0 to 890  
Data columns (total 3 columns):  
Pclass      891 non-null int64  
Age         891 non-null float64  
Sex         891 non-null object  
dtypes: float64(1), int64(1), object(1)  
memory usage: 21.0+ KB
```

#数据分割

```
x_train, x_test, y_train, y_test = train_test_split(x, y, random_state=22)  
x_train.head()
```

	Pclass	Age	Sex
635	2	28.000000	female
331	1	45.500000	male
332	1	38.000000	male
250	3	29.699118	male
702	3	18.000000	female

数据处理

#字典特征提取

dc=DictVectorizer(sparse=False) #返回完整矩阵

new_x_train=dc.fit_transform(x_train.to_dict(orient='records'))

new_x_test=dc.transform(x_test.to_dict(orient='records'))

new_x_train

```
array([[28. , 2. , 1. , 0. ],
       [45.5, 1. , 0. , 1. ],
       [38. , 1. , 0. , 1. ],
       ...,
       [35. , 2. , 0. , 1. ],
       [47. , 3. , 1. , 0. ],
       [39. , 3. , 1. , 0. ]])
```

模型建立

```
rf=RandomForestClassifier()  
#配合网格搜索  
param_dict={"n_estimators": [120, 200, 300, 500, 800, 1200], "max_depth": [5, 8, 15, 25, 30]}  
#实例化GridSearchCV  
gs=GridSearchCV(rf, param_grid=param_dict, cv=3)  
gs.fit(new_x_train, y_train)  
#输出最优参数组合  
gs.best_params_
```

```
{'max_depth': 5, 'n_estimators': 120}
```

```
#使用最优参数组合重新训练模型
```

```
rf2=RandomForestClassifier(n_estimators=300, max_depth=5)  
rf2.fit(new_x_train, y_train)
```

```
RandomForestClassifier(bootstrap=True, class_weight=None, criterion='gini',  
                        max_depth=5, max_features='auto', max_leaf_nodes=None,  
                        min_impurity_decrease=0.0, min_impurity_split=None,  
                        min_samples_leaf=1, min_samples_split=2,  
                        min_weight_fraction_leaf=0.0, n_estimators=300,  
                        n_jobs=None, oob_score=False, random_state=None,  
                        verbose=0, warm_start=False)
```

模型评估

```
print("训练集:", rf2.score(new_x_train, y_train))  
print("测试集:", rf2.score(new_x_test, y_test))
```

训练集: 0.8517964071856288

测试集: 0.7623318385650224

小结

本节我们主要学习了bagging和随机森林的原理：

■ bagging融合框架：

- 同质学习 bagging + XXX算法，**并行**框架
- 有放回抽样制造**数据差异化**，进而实现模型差异化
- 追求**低方差**

■ 随机森林：

- 基于bagging框架
- 底层是cart树
- **样本随机**和**特征随机**
- **泛化能力强**，稳



1
2
chapter

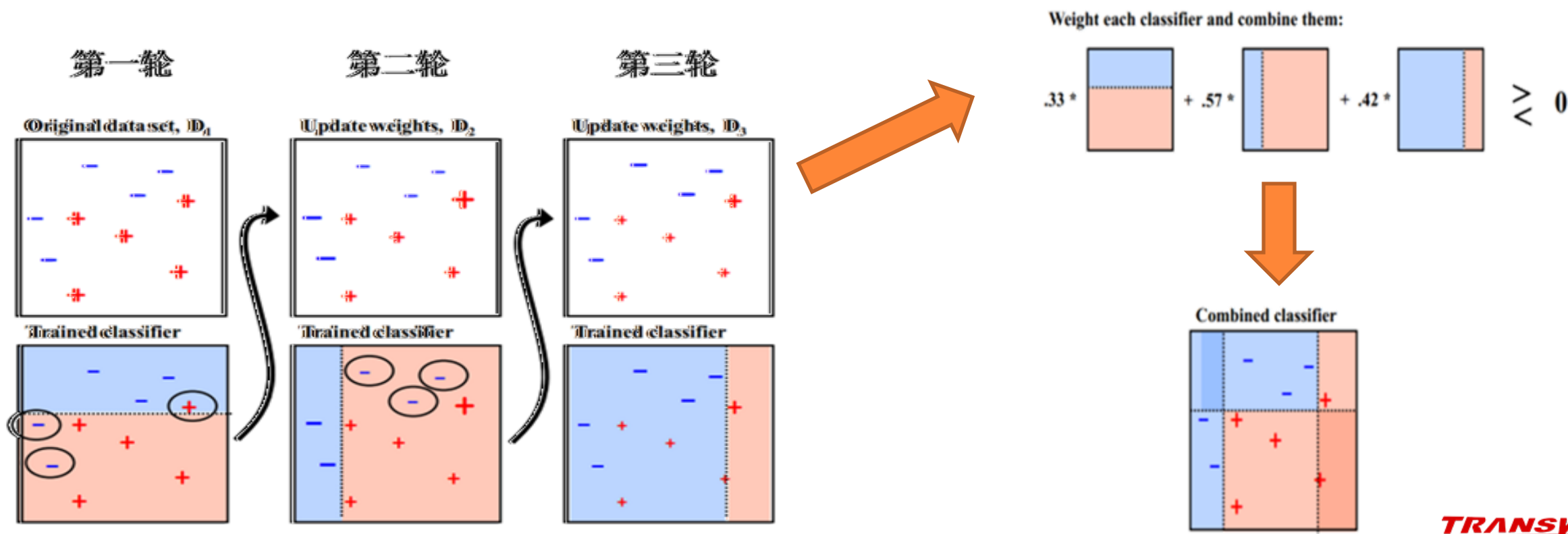
Boosting 和 GBDT

Boosting 原理

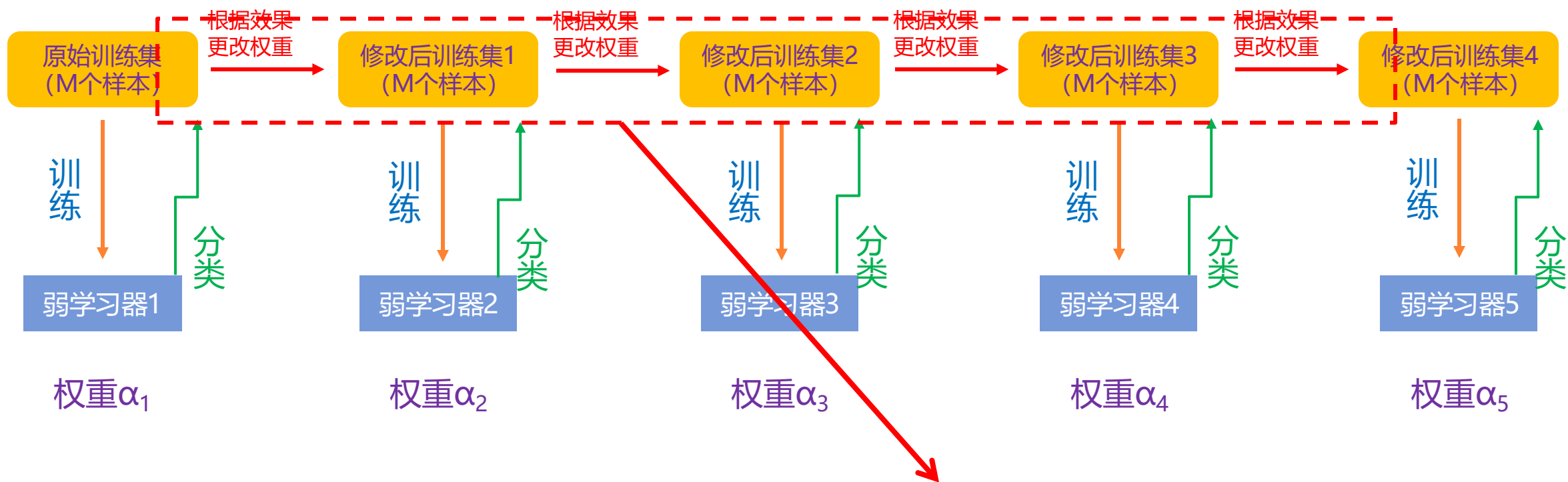
- **错题本机制**: Boosting 融合在每一次训练模型的时候都更加**关注**上一次模型**错判**的样例, 会给这些错判的样例**更大**的权重, 这样做的目的就是为了**加强**对错判样本的学习, 让模型通过不断的迭代, 效果越来越好
- **线性加权**: 最终将多次迭代训练得到的弱模型进行**加权求和**, 得到最终的强模型
- **典型算法**: Adaboost、GBDT、XGBoost、LightGBM、CatBoost

Boosting 原理 - 错题本机制 (Adaboost)

- AdaBoost 是 Adaptive boosting 的缩写，从名字可以看出来，它是一种基于 Boosting 融合框架的算法。下图展现了 Adaboost 算法不断迭代的过程：

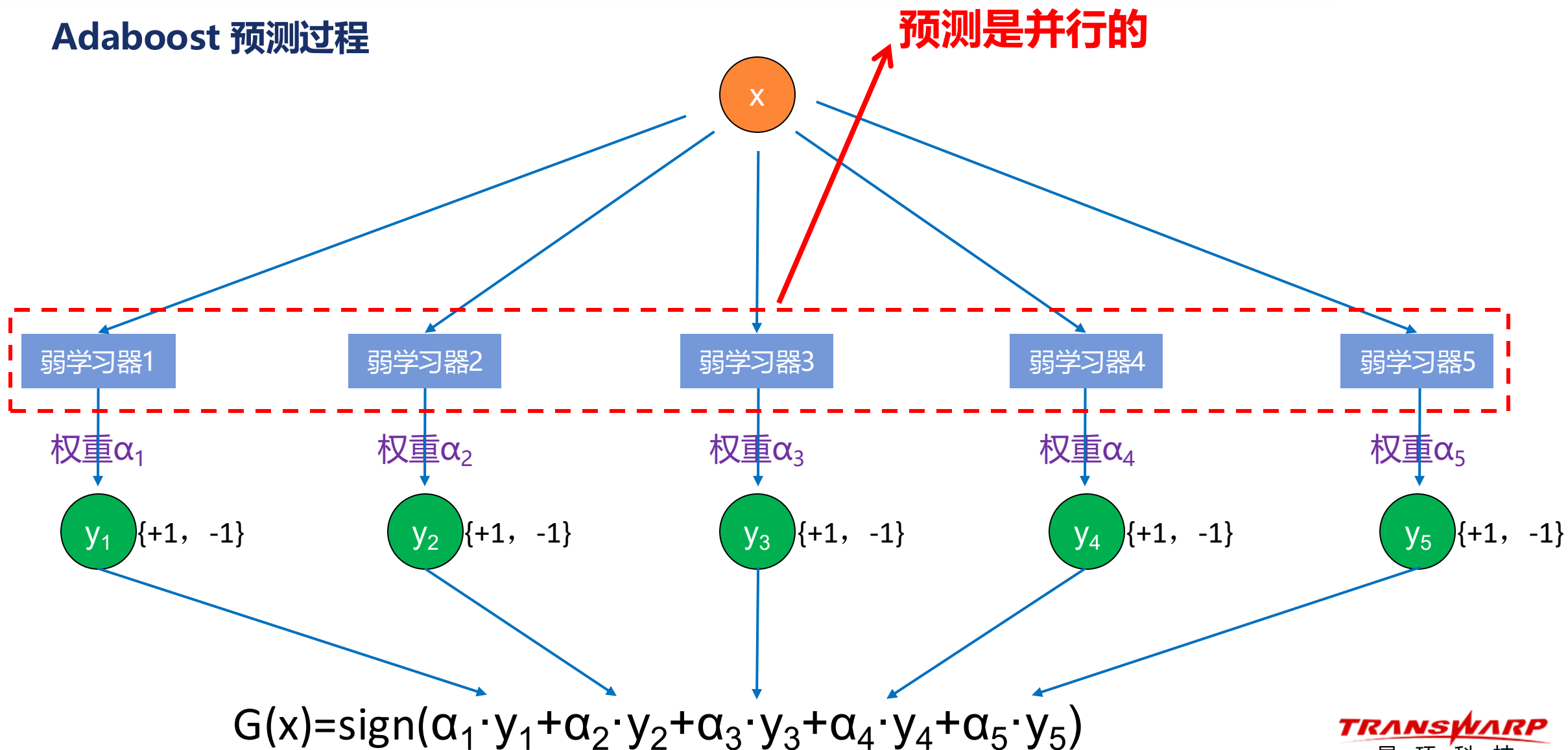


Adaboost 训练过程



串行: 上一个模型训练完, 下一个模型才能训练

Adaboost 预测过程



$$Z_m = \sum_{i=1}^n w_m^i e^{-\alpha_m y^{(i)} G_m(x^{(i)})}$$

Adaboost 构建过程：两个权重

初始化样本权重

$$D_1 = (w_1^1, w_1^2, \dots, w_1^i, \dots, w_1^n), w_1^i = \frac{1}{n}$$

$$D_{m+1} = (w_{m+1}^1, w_{m+1}^2, \dots, w_{m+1}^i, \dots, w_{m+1}^n)$$

$$w_{m+1}^i = \frac{w_m^i}{Z_m} e^{-\alpha_m y^{(i)} G_m(x^{(i)})}$$

修改样本权重

m轮训练集
(n个样本)

m+1轮训练集1
(n个样本)

训练

弱学习器 G_m

预测并计算
错误率 ϵ_m

$$\begin{aligned} \epsilon_m &= P(G_m(x^{(i)}) \neq y^{(i)}) \\ &= \sum_{i=1}^n w_m^i I(G_m(x^{(i)}) \neq y^{(i)}) \end{aligned}$$

$$G_m(x): x \rightarrow \{-1, +1\}$$

$$f(x) = \sum_{m=1}^M \alpha_m G_m(x)$$

根据错误率 ϵ_m 计算模型权重 α_m

$$\alpha_m = \frac{1}{2} \ln \frac{1 - \epsilon_m}{\epsilon_m}$$

$$G(x) = \text{sign}(f(x)) = \text{sign}\left(\sum_{m=1}^M \alpha_m G_m(x)\right)$$

Adaboost 构建过程：两个权重

■ 模型权重：

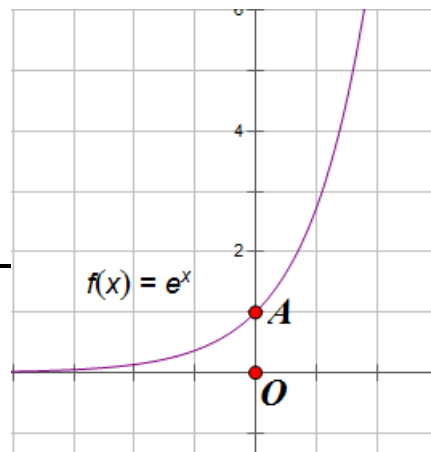
$$\alpha_m = \frac{1}{2} \ln \frac{1 - \varepsilon_m}{\varepsilon_m}$$

- 错误率越小，权重越大

■ 样本权重：

$$D_{m+1} = (w_{m+1}^1, w_{m+1}^2, \dots, w_{m+1}^i, \dots, w_{m+1}^n), w_{m+1}^i = \frac{w_m^i}{Z_m} e^{-\alpha_m y^{(i)} G_m(x^{(i)})}$$

- 分类正确，权重减小
- 分类错误，权重增大



m轮下标为i的样本权重

错误为1，正确为0

$$\varepsilon_m = P(G_m(x^{(i)} \neq y^{(i)}) = \sum_{i=1}^n w_m^i I(G_m(x^{(i)}) \neq y^{(i)})$$

m轮错误样本权重之和，0-1

m轮下标为i的样本预测值{-1,+1}

下标为i的样本真实值{-1,+1}

Adaboost 算法示例

■ 权重训练用下列训练样本，试用 AdaBoost 算法学习一个强分类器

序号	1	2	3	4	5	6	7	8	9	10
X	0	1	2	3	4	5	6	7	8	9
Y	1	1	1	-1	-1	-1	1	1	1	-1

■ 弱分类器

- $G_1(x) = \begin{cases} 1 & \text{if } x < 2.5 \\ -1 & \text{otherwise} \end{cases}$
- $G_2(x) = \begin{cases} 1 & \text{if } x < 8.5 \\ -1 & \text{otherwise} \end{cases}$
- $G_3(x) = \begin{cases} 1 & \text{if } x \geq 5.5 \\ -1 & \text{otherwise} \end{cases}$

Adaboost 算法示例

- 初始化训练数据集的权值分布

$$D_1 = (w_{11}, w_{12}, \dots, w_{1i}, \dots, w_{1n}), \quad w_{1i} = \frac{1}{N}, \quad i = 1, 2, \dots, N$$

$$w_{1i} = 0.1$$

Adaboost 算法示例 – 第一轮迭代

1. 预测结果 pred_1

$$\text{pred}_1 = [1, 1, 1, -1, -1, -1, -1, -1, -1, -1]$$

2. 误差 ϵ_1

错误样本: $x = 6, 7, 8$

$$\epsilon_1 = D[6] + D[7] + D[8] = 0.1 + 0.1 + 0.1 = 0.3$$

3. 分类器权重 α_1

$$\alpha_1 = 0.5 \cdot \ln\left(\frac{1 - \epsilon_1}{\epsilon_1}\right) = 0.5 \cdot \ln\left(\frac{0.7}{0.3}\right) \approx 0.4236$$

Adaboost 算法示例 – 第一轮迭代结果

- 正确样本: $x = 0, 1, 2, 3, 4, 5, 9 \rightarrow D_i \cdot e^{-\alpha_1}$
- 错误样本: $x = 6, 7, 8 \rightarrow D_i \cdot e^{\alpha_1}$
- 归一化后:

$$D \approx [0.0714, 0.0714, 0.0714, 0.0714, 0.0714, 0.0714, 0.1667, 0.1667, 0.1667, 0.0714]$$

$$f_1(x) = 0.4236G_1(x)$$

Adaboost 算法示例 – 第二轮迭代

1. 预测结果 pred_2

$$\text{pred}_2 = [1, 1, 1, 1, 1, 1, 1, 1, -1]$$

2. 误差 ϵ_2

错误样本: $x = 3, 4, 5$

$$\epsilon_2 = D[3] + D[4] + D[5] \approx 0.0714 \times 3 = 0.2143$$

3. 分类器权重 α_2

$$\alpha_2 = 0.5 \cdot \ln\left(\frac{1 - \epsilon_2}{\epsilon_2}\right) = 0.5 \cdot \ln\left(\frac{0.7857}{0.2143}\right) \approx 0.6496$$

Adaboost 算法示例 – 第二轮迭代结果

◆ 更新数据集的权值分布

$D \approx [0.0455, 0.0455, 0.0455, 0.1667, 0.1667, 0.1667, 0.1061, 0.1061, 0.1061, 0.0455]$

$$f_2(x) = 0.4236G_1(x) + 0.6496G_2(x)$$

◆ 分类器 $\text{sign}(f_2(x))$ 在训练数据集上有 3 个误分类点

Adaboost 算法示例 – 第三轮迭代

1. 预测结果 pred_3

$$\text{pred}_3 = [-1, -1, -1, -1, -1, -1, 1, 1, 1, 1]$$

2. 误差 ϵ_3

错误样本: $x = 0, 1, 2, 9$

$$\epsilon_3 = D[0] + D[1] + D[2] + D[9] \approx 0.0455 \times 4 = 0.1818$$

3. 分类器权重 α_3

$$\alpha_3 = 0.5 \cdot \ln\left(\frac{1 - \epsilon_3}{\epsilon_3}\right) = 0.5 \cdot \ln\left(\frac{0.8182}{0.1818}\right) \approx 0.7520$$

Adaboost 算法示例 – 第三轮迭代结果

◆ 更新数据集的权值分布

$$D \approx [0.125, 0.125, 0.125, 0.1019, 0.1019, 0.1019, 0.0648, 0.0648, 0.0648, 0.125]$$

$$f_3(x) = 0.4236G_1(x) + 0.6496G_2(x) + 0.7520G_3(x)$$

◆ 分类器 $\text{sign}(f_3(x))$ 在训练数据集上有 4 个误分类点

Adaboost 算法示例 - 最终强分类器

$$F(x) = \text{sign} (0.4236 \cdot G_1(x) + 0.6496 \cdot G_2(x) + 0.7520 \cdot G_3(x))$$

Adaboost – 代码示例

```
# 加载 iris 数据集
```

```
iris = datasets.load_iris()
```

```
X = iris.data
```

```
y = iris.target
```

```
# 将数据集分为训练集和测试集
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# 创建一个决策树桩作为弱分类器
```

```
dt_stump = DecisionTreeClassifier(max_depth=1, min_samples_leaf=1)
```

```
# 创建 Adaboost 分类器，使用决策树桩作为基分类器
```

```
ada = AdaBoostClassifier(estimator=dt_stump, n_estimators=50, learning_rate=1, random_state=42)
```

```
# 训练 Adaboost 模型
```

```
ada.fit(X_train, y_train)
```

```
# 在测试集上进行预测
```

```
y_pred = ada.predict(X_test)
```

```
# 计算准确率
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
# 绘制分类决策边界（仅适用于二维数据，这里为了简化，我们使用前两个特征）
```

```
X_train_2d = X_train[:, :2]
```

```
X_test_2d = X_test[:, :2]
```

```
ada_2d = AdaBoostClassifier(estimator=dt_stump, n_estimators=50, learning_rate=1, random_state=42)
```

```
ada_2d.fit(X_train_2d, y_train)
```

```
# 创建网格点以绘制决策边界
```

```
x_min, x_max = X_train_2d[:, 0].min() - 1, X_train_2d[:, 0].max() + 1
```

```
y_min, y_max = X_train_2d[:, 1].min() - 1, X_train_2d[:, 1].max() + 1
```

```
xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.1), np.arange(y_min, y_max, 0.1))
```

```
# 预测网格点的类别
```

```
Z = ada_2d.predict(np.c_[xx.ravel(), yy.ravel()])
```

```
Z = Z.reshape(xx.shape)
```

```
# 绘制决策边界和数据点
```

```
plt.contourf(xx, yy, Z, alpha=0.8)
```

```
plt.scatter(X_train_2d[:, 0], X_train_2d[:, 1], c=y_train, edgecolor='k', s=20)
```

```
plt.xlabel('Sepal length')
```

```
plt.ylabel('Sepal width')
```

```
plt.title('Adaboost Decision Boundary')
```

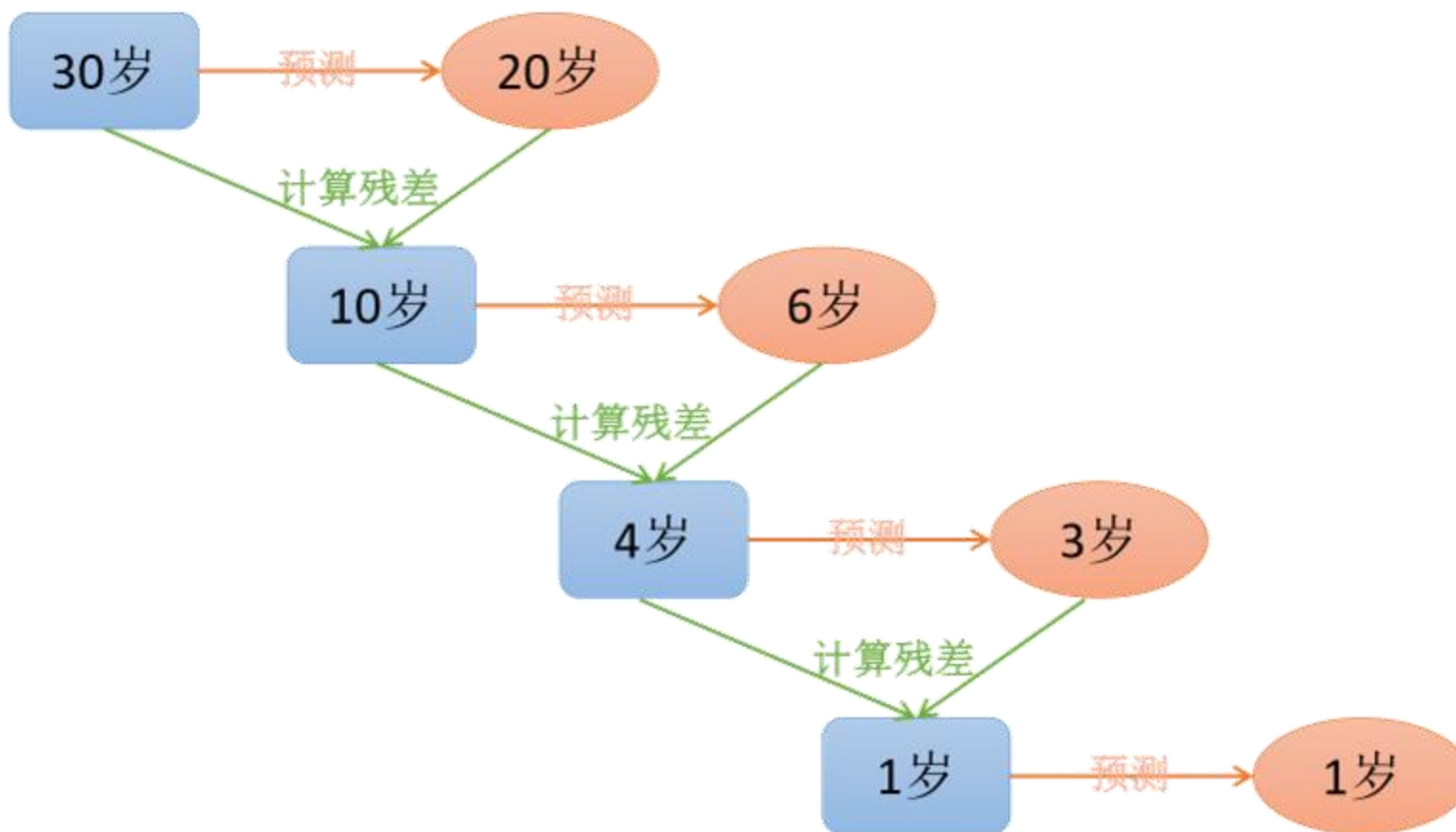
```
plt.show()
```

Bagging 和 Boosting 区别

- **数据差异化**: bagging: 有放回抽样; boosting: 修改样本权重
- **投票**: bagging: **平权**投票; boosting: **加权**投票
- **训练方式**: bagging: **并行**; boosting: **串行**
- **追求目标**: bagging: 解决过拟合, 追求**低方差**; boosting: 解决欠拟合, 追求**低偏差**
- **子模型**
 - Bagging 子模型都是**强**模型(过拟合的)
 - Boosting 子模型都是**弱**模型(欠拟合的)

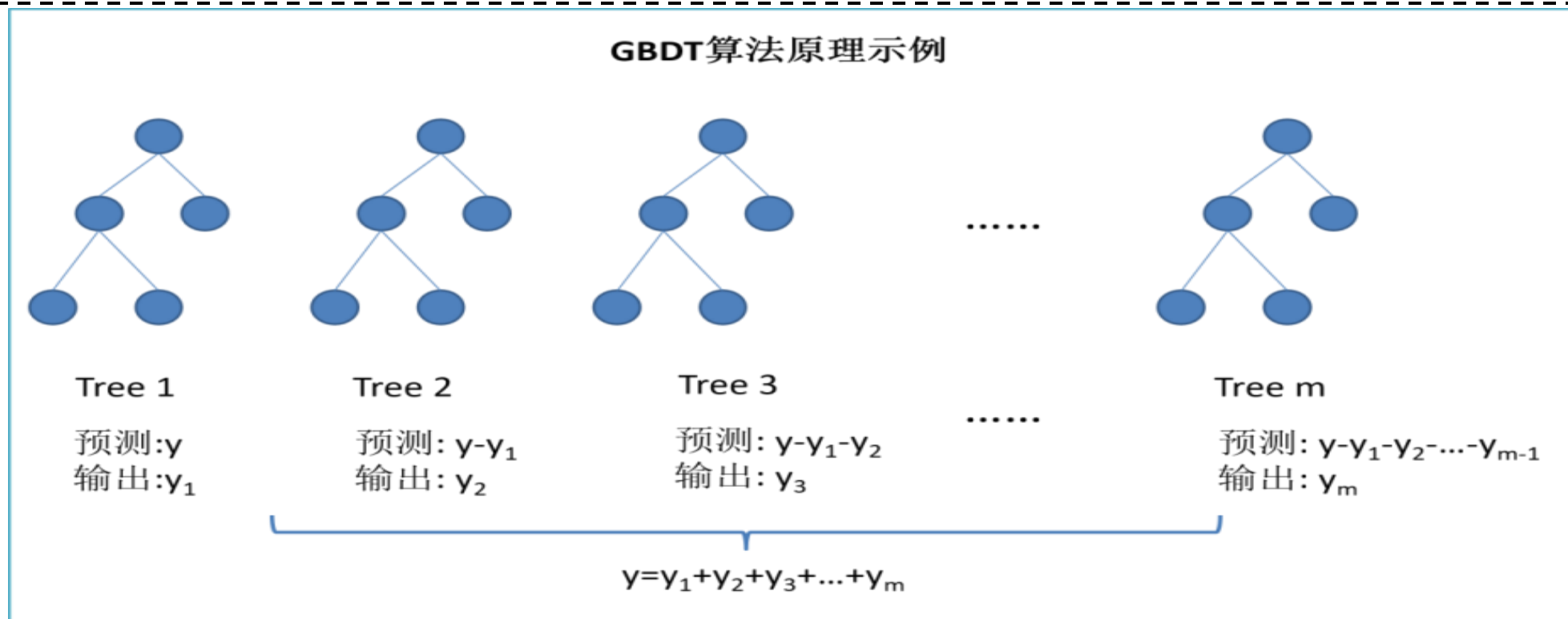
GBDT 代码层面实现原理

- 不断的拟合残差，训练多颗 cart 回归树



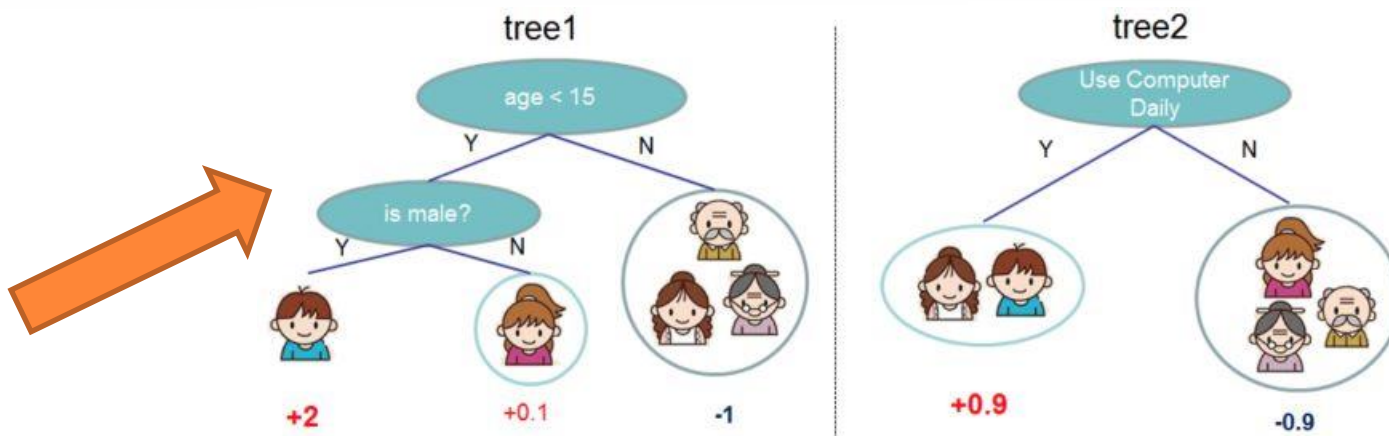
GBDT 代码层面实现原理

- 总共构造了 m 个cart树子模型，每一个模型都是对上一轮模型的预测**残差**进行学习（以残差作为**目标变量**），最终 m 个子模型预测结果**简单相加**就是目标变量 y 的**预测值**。



GBDT 代码层面实现原理

样本\特征	Age	is male	Use computer daily	Like CG
	79	1	0	?
	75	0	0	?
	29	0	1	?
	5	1	1	?
	8	0	0	?



$$f(\text{Young man icon}) = 2 + 0.9 = 2.9 \quad f(\text{Old man icon}) = -1 - 0.9 = -1.9$$

GBDT 数学层面原理 -- 在函数空间用梯度下降(一阶泰勒展开)优化

- **GB (Gradient Boosting)** : **梯度提升** - 在函数空间用梯度下降来优化损失函数,让损失函数值变小,模型得到提升;
- **DT**: cart 回归树
- **Shrinkage**: **学习率** - 既然是梯度下降,是不是就有学习率,所以并不是完全的拟合残差,只拟合 $0.1 \times (\text{学习率}) \times \text{残差} (\text{梯度})$

GBDT 数学层面原理 -- 参数空间

■ 参数空间的梯度下降

$$\theta^t = \theta^{t-1} - \alpha \cdot g^{t-1}$$

$$\begin{aligned}\theta^0 &= c \\ \theta^1 &= \theta^0 - \alpha \cdot g^0 \\ \theta^2 &= \theta^1 - \alpha \cdot g^1 \\ &\vdots \\ \theta^{t-1} &= \theta^{t-2} - \alpha \cdot g^{t-2} \\ \theta^t &= \theta^{t-1} - \alpha \cdot g^{t-1}\end{aligned}$$

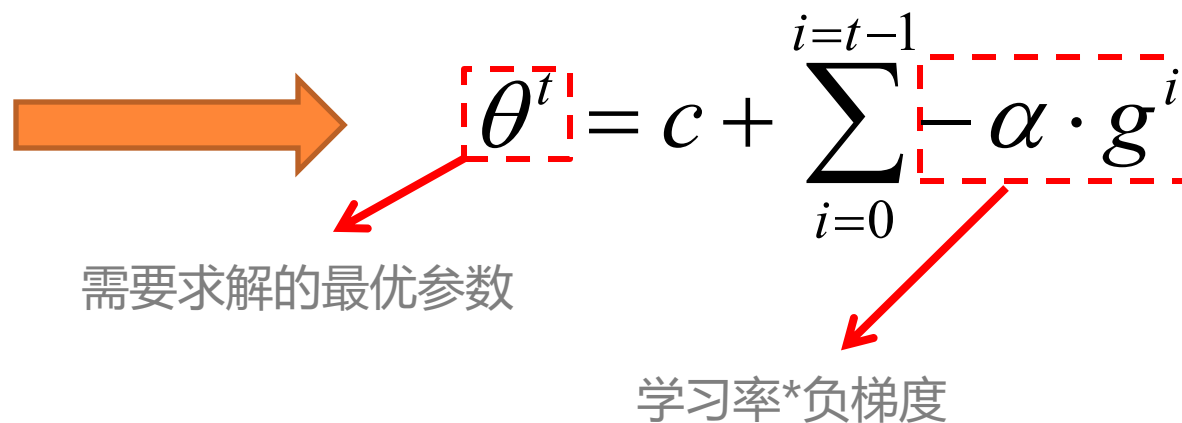


Diagram illustrating the iterative update of parameters in GBDT. An orange arrow points from the sequence of equations on the left to the final equation on the right. The final equation is $\theta^t = c + \sum_{i=0}^{t-1} -\alpha \cdot g^i$. A red dashed box highlights the entire right-hand side of this equation. A red arrow points from the text "需要求解的最优参数" (Optimal parameter to be solved) to the θ^t term. Another red arrow points from the text "学习率*负梯度" (Learning rate * negative gradient) to the $-\alpha \cdot g^i$ term inside the summation.

$$\theta^t = c + \sum_{i=0}^{t-1} -\alpha \cdot g^i$$

需要求解的最优参数

学习率*负梯度

GBDT 数学层面原理 -- 函数空间

■ 函数空间的梯度下降

$$\theta^t = \theta^{t-1} - \alpha \cdot g^{t-1}$$

$$F^t(x) = F^{t-1}(x) + f^{t-1}(x)$$

$$f^{t-1}(x) = -\alpha \cdot \frac{\partial L(y, F(x))}{\partial F^{t-1}(x)}$$

$$F^0(x) = c$$

$$F^1(x) = F^0(x) + f^0(x)$$

$$F^2(x) = F^1(x) + f^1(x)$$

$$\vdots$$

$$F^{t-1}(x) = F^{t-2}(x) + f^{t-2}(x)$$

$$F^t(x) = F^{t-1}(x) + f^{t-1}(x)$$



$$F^t(x) = c + \sum_{i=0}^{t-1} f^i(x)$$

以负梯度为目标拟合一颗cart回归树

需要求解的最优函数

$$= c + \sum_{i=0}^{t-1} -\alpha \cdot \frac{\partial L(y, F(x))}{\partial F^i(x)}$$

学习率*负梯度 (对函数求导)

GBDT 数学层面原理 -- 回归算法流程

输入：训练数据集

$$T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}, x_i \in \mathcal{X} \subseteq R^n, y_i \in \mathcal{Y} \subseteq R, i = 1, 2, \dots, N$$

，最大迭代次数 M ，损失函数 $L(y, f(x))$

输出：回归树 $\hat{f}(x)$

1. 初始化

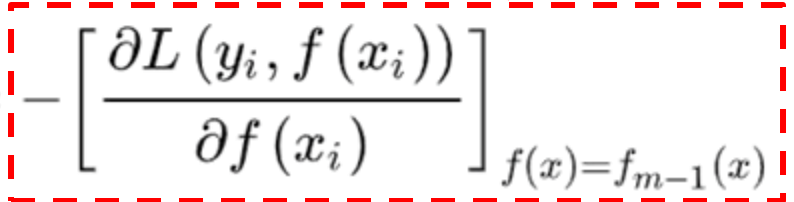
第一个常数函数 c

$$f_0(x) = \arg \min_c \sum_{i=1}^N L(y_i, c)$$


2. 对 $m = 1, 2, \dots, M$

2.1 对 $i = 1, 2, \dots, N$ 计算

计算**负梯度**，一般**是残差**

$$r_{mi} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x)=f_{m-1}(x)}$$


GBDT 数学层面原理 -- 回归算法流程

2.2 对 r_{mi} 拟合回归树, 得到第 m 棵树的叶结点区域 $R_{mj}, j = 1, 2, \dots, J$

2.3 对 $j = 1, 2, \dots, J$ 计算

$$c_{mj} = \arg \min_c \sum_{x_i \in R_{mj}} L(y_i, f_{m-1}(x_i) + c)$$

第m颗树第j个叶子节点的预测值

2.4 更新 $f_m(x) = f_{m-1}(x) + \sum_{j=1}^J c_{mj} I(x \in R_{mj})$

拟合一棵回归树

3. 得到回归树

$$\hat{f}(x) = f_M(x) = \sum_{m=1}^M \sum_{j=1}^J c_{mj} I(x \in R_{mj})$$

所有树的预测值之和

GBDT 数学层面原理 -- 常见损失函数（分类）

- **指数损失（二分类）**：此时就是adaboost算法，底层是分类树

$$L(y, f(x)) = \exp(-yf(x))$$

- **对数似然损失函数（二分类）**：

$$L(y, f(x)) = \log(1 + \exp(-yf(x)))$$

- **交叉熵损失（多分类）**：

$$L(y, f(x)) = - \sum_{i=1}^K y_i \log p_i(x)$$

GBDT 数学层面原理 -- 常见损失函数（回归）

■ 均方差损失（L2）：

$$L(y, f(x)) = (y - f(x))^2$$

■ 绝对损失（L1）：

$$L(y, f(x)) = |y - f(x)|$$

■ Huber损失：

$$L(y, f(x)) = \begin{cases} \frac{1}{2} (y - f(x))^2 & |y - f(x)| \leq \delta \\ \delta (|y - f(x)| - \frac{\delta}{2}) & |y - f(x)| > \delta \end{cases}$$

导入工具包

```
import pandas as pd
import numpy as np
from sklearn.feature_extraction import DictVectorizer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingClassifier
```

读取数据

```
data=pd.read_csv("train.csv")
data.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

#选择x和y

```
x=data[['Pclass','Age','Sex']]
y=data['Survived']
x.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 3 columns):
Pclass    891 non-null int64
Age       714 non-null float64
Sex       891 non-null object
dtypes: float64(1), int64(1), object(1)
memory usage: 21.0+ KB
```

数据处理

#填充缺失值

```
x = x.fillna(x.mean())  
x.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 891 entries, 0 to 890  
Data columns (total 3 columns):  
Pclass      891 non-null int64  
Age         891 non-null float64  
Sex         891 non-null object  
dtypes: float64(1), int64(1), object(1)  
memory usage: 21.0+ KB
```

#数据分割

```
x_train, x_test, y_train, y_test = train_test_split(x, y, random_state=22)  
x_train.head()
```

	Pclass	Age	Sex
635	2	28.000000	female
331	1	45.500000	male
332	1	38.000000	male
250	3	29.699118	male
702	3	18.000000	female

数据处理

#字典特征提取

dc=DictVectorizer(sparse=False) #返回完整矩阵

new_x_train=dc.fit_transform(x_train.to_dict(orient='records'))

new_x_test=dc.transform(x_test.to_dict(orient='records'))

new_x_train

```
array([[28. , 2. , 1. , 0. ],
       [45.5, 1. , 0. , 1. ],
       [38. , 1. , 0. , 1. ],
       ...,
       [35. , 2. , 0. , 1. ],
       [47. , 3. , 1. , 0. ],
       [39. , 3. , 1. , 0. ]])
```

模型建立与评估

```
gbdt=GradientBoostingClassifier(n_estimators=200, learning_rate=0.1, max_depth=5)
gbdt.fit(new_x_train, y_train)
```

```
GradientBoostingClassifier(criterion='friedman_mse', init=None,
                           learning_rate=0.1, loss='deviance', max_depth=5,
                           max_features=None, max_leaf_nodes=None,
                           min_impurity_decrease=0.0, min_impurity_split=None,
                           min_samples_leaf=1, min_samples_split=2,
                           min_weight_fraction_leaf=0.0, n_estimators=200,
                           n_iter_no_change=None, presort='auto',
                           random_state=None, subsample=1.0, tol=0.0001,
                           validation_fraction=0.1, verbose=0,
                           warm_start=False)
```

#模型评估

```
print("训练集:", gbdt.score(new_x_train, y_train))
print("测试集:", gbdt.score(new_x_test, y_test))
```

训练集: 0.8907185628742516

测试集: 0.7847533632286996

XGBoost -- 在函数空间用牛顿法(二阶泰勒展开)优化

- **牛顿法**：二阶泰勒展开（一阶导和二阶导），优化速度更快
- **正则化**：在优化过程中XGBoost为防止过拟合，在损失函数中加入**惩罚项**（叶子节点个数和叶子节点分值平方和）
- **并行化**：子集分割并行和特征选择并行
- **预排序**：预先对特征进行排序并存入内存，加速后续训练
- **随机**：参考随机森林，**样本随机**和**特征随机**，提高泛化能力

小结

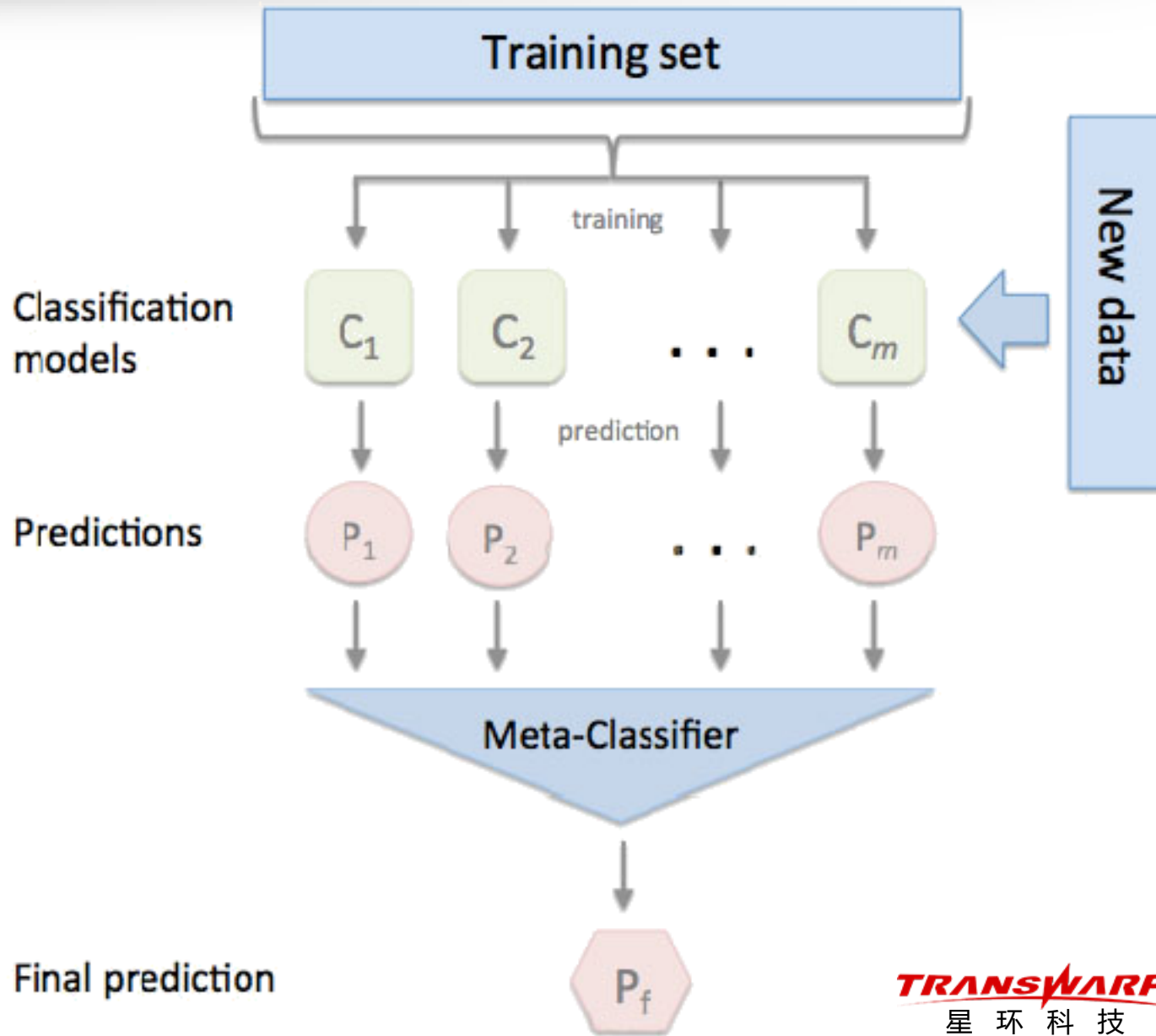
- **牛顿法**：二阶泰勒展开（一阶导和二阶导），优化速度更快
- **正则化**：在优化过程中 XGBoost 为防止过拟合，在损失函数中加入**惩罚项**（叶子节点个数和叶子节点分值平方和）
- **并行化**：子集分割并行和特征选择并行
- **预排序**：预先对特征进行排序并存入内存，加速后续训练
- **随机**：参考随机森林，**样本随机**和**特征随机**，提高泛化能力

1
3
chapter

Stacking

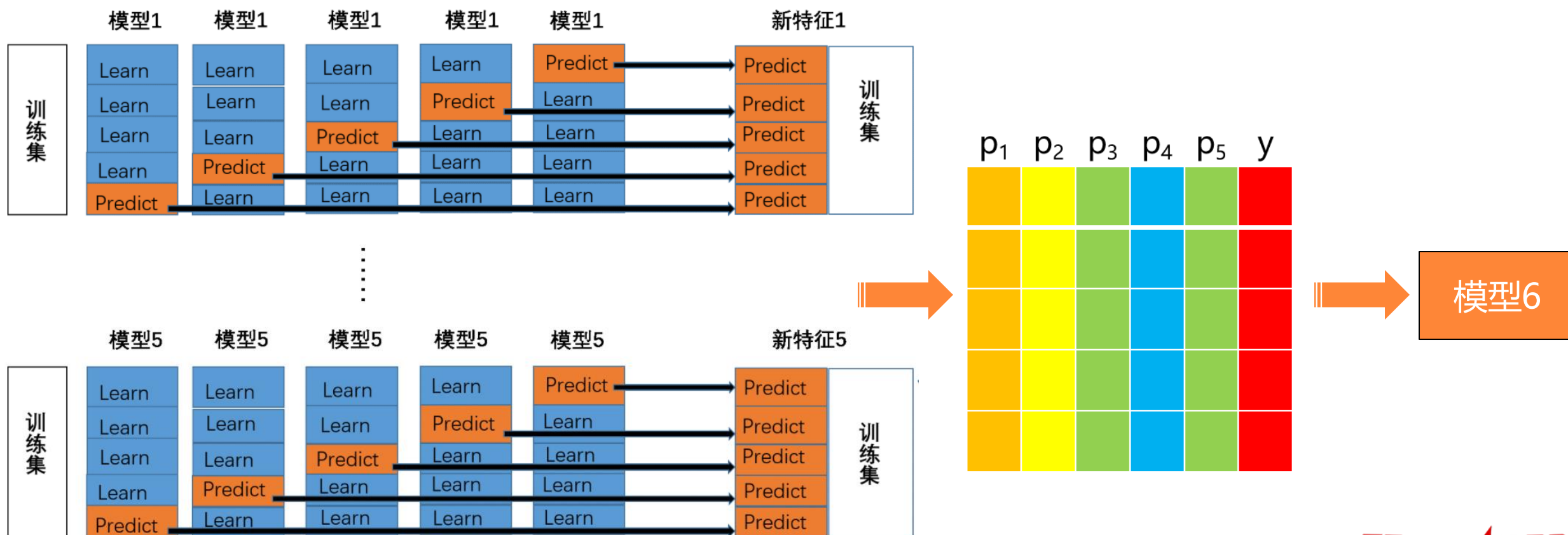
Stacking 原理导读

Stacking 模型是指将**多种算法模型**组合在一起取得更好表现的一种**集成学习**模型。一般情况下，Stacking 模型分为**两层**。第一层中我们**训练多个不同的模型**，然后再以第一层训练的各个模型的**输出作为输入**来训练第二层的模型，以得到一个**最终的输出**。



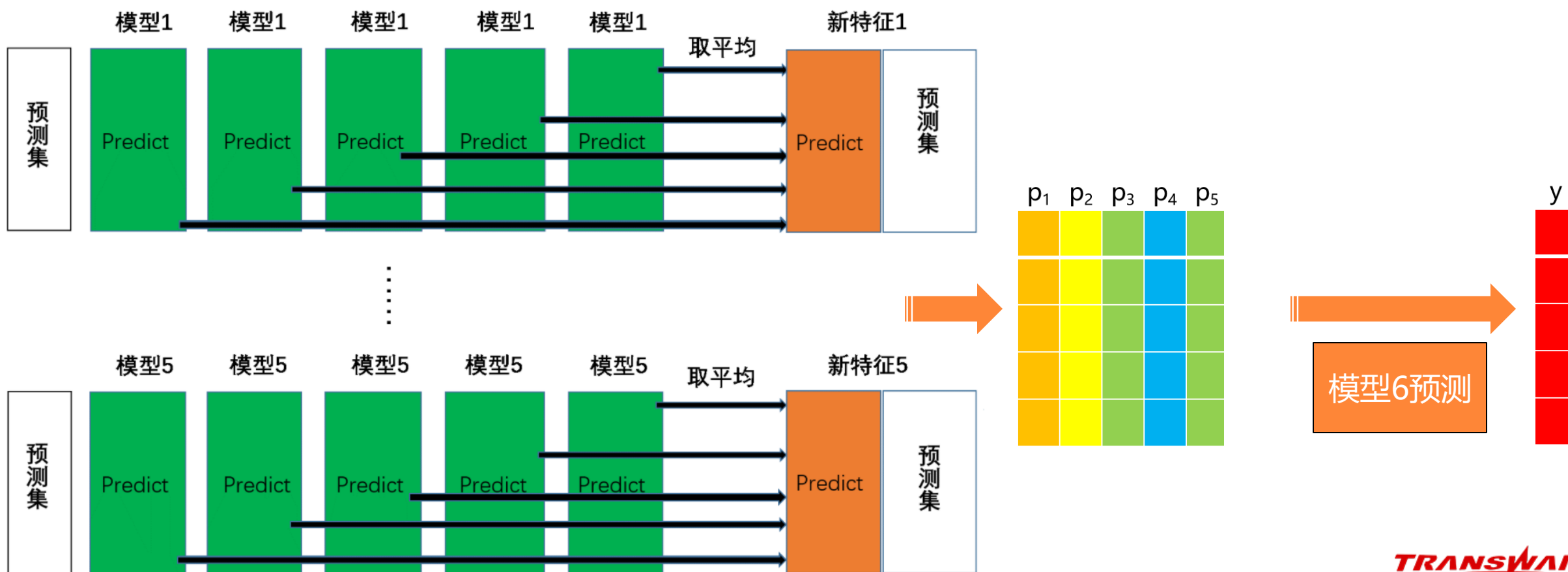
训练集特征提取

- 训练集做**k折**拆分。每一份用一个模型预测



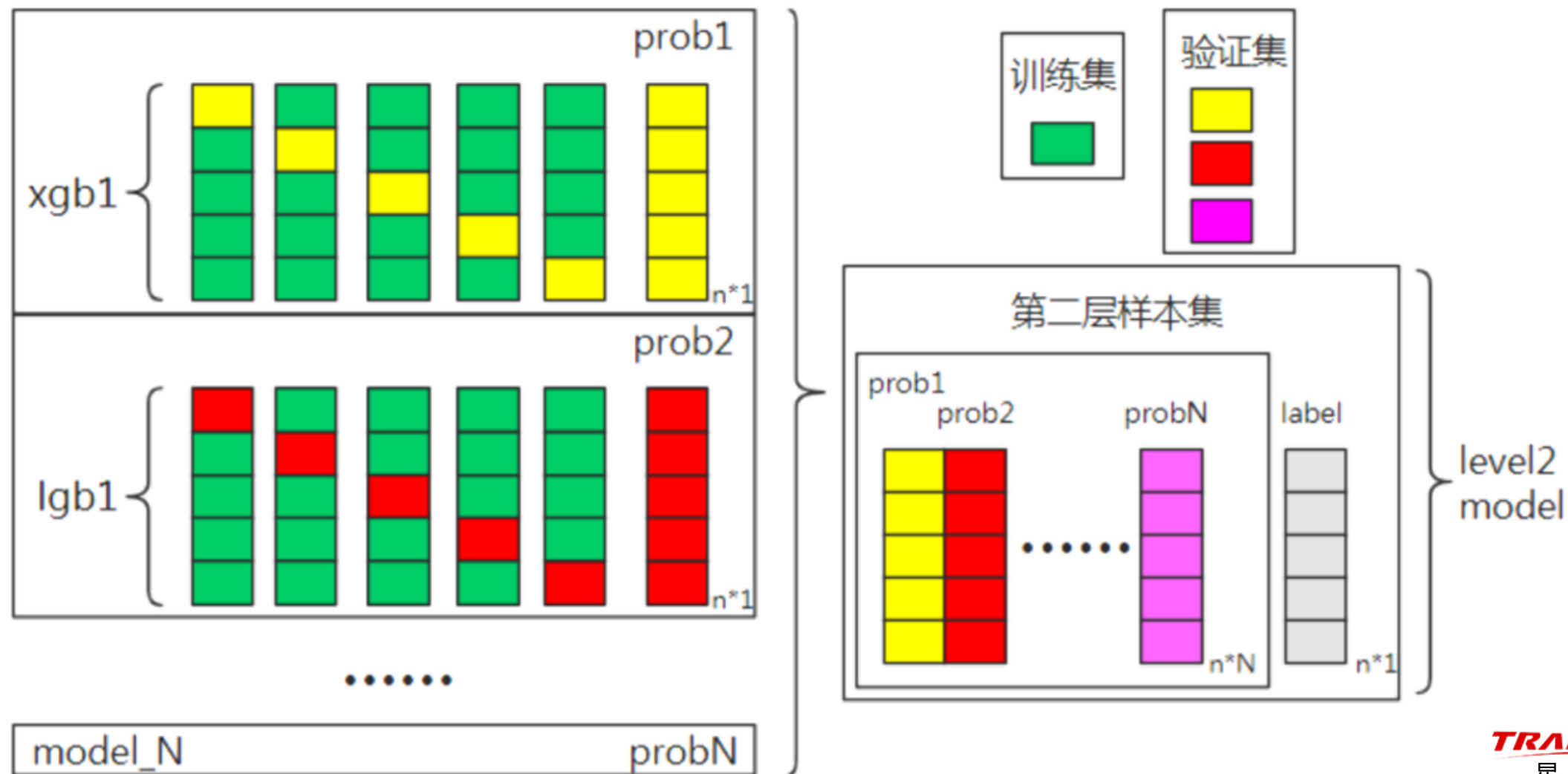
测试集特征提取

■ 测试集**取平均**;



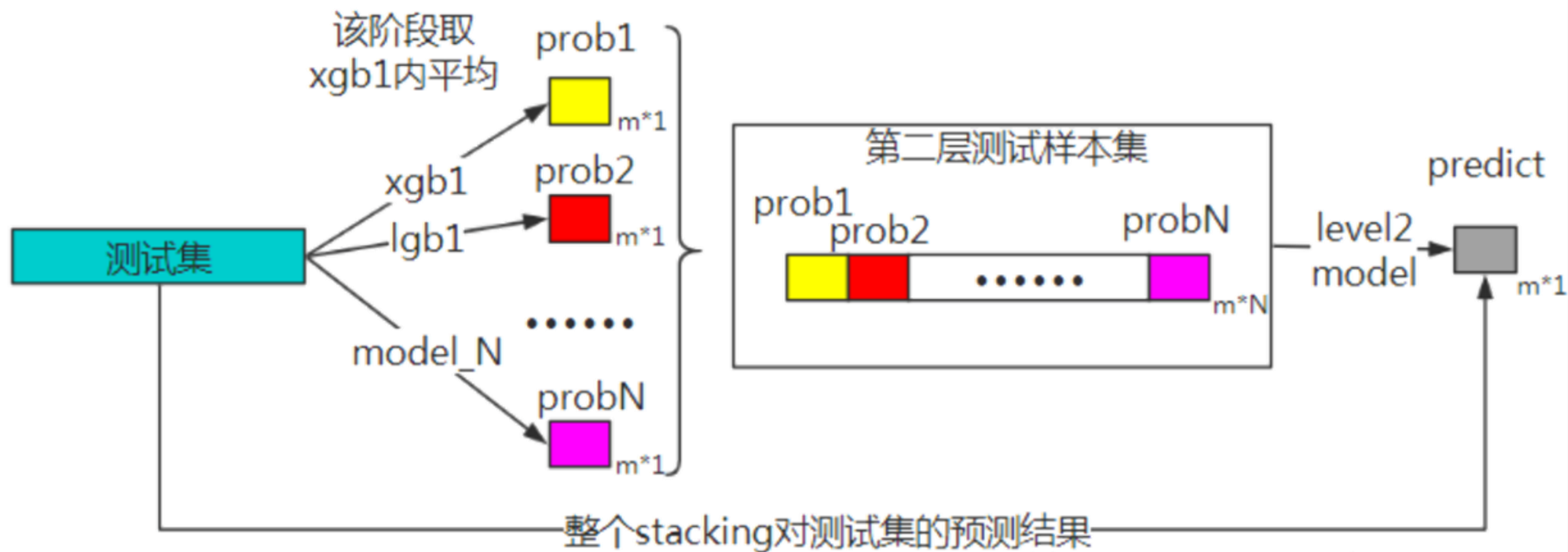
训练阶段

stacking训练阶段



测试阶段

stacking测试阶段



随机森林中的随机是指（ ）

- A. 随便构建树模型
- B. 随机选择一个树模型
- C. 随机选择多个树模型
- D. 在构建每个树模型时随机选择样本和特征

Adaboost 的核心思想是（ ）

- A. 给定一个弱学习算法和一个训练集，将该学习算法使用多次，得出预测函数序列，进行投票；
- B. 针对同一个训练集训练不同的弱分类器并集合起来，构成一个强分类器；
- C. 利用多棵树对样本进行训练并预测的一种分类器；
- D. 基于前向策略的加法模型，每阶段使用一个基模型去拟合上一阶段基模型的残差。

下列关于 Bagging 的说法错误的是（ ）

- A. 各基分类器之间有较强的依赖，不可以进行并行训练；
- B. 最著名的算法之一是基于决策树基分类器的随机森林；
- C. 当训练样本数量较少时，子集之间可能有重叠；
- D. 为了让基分类器之间互相独立，需要将训练集分为若干子集；

小明使用 Stacking 模型融合框架训练模型，选取了 7 种不同的算法模型，并进行 5 折交叉验证，请问在预测时，对于同一个样本需要多少次模型的预测（ ）

- A. 8
- B. 40
- C. 36
- D. 35

导入工具包

```
#安装mlxtend
pip install mlxtend

#导入依赖包
import numpy as np
import matplotlib.pyplot as plt
from mlxtend.regressor import StackingRegressor
from mlxtend.data import boston_housing_data
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.svm import SVR
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import GridSearchCV
```

数据与模型准备

```
x, y = boston_housing_data()
# 划分数据集
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2)

#初始化基模型
lr = LinearRegression()
svr_lin = SVR(kernel='linear', gamma='auto')
svr_rbf = SVR(kernel='rbf', gamma='auto')
ridge = Ridge(random_state=2021)
models = [lr, svr_lin, svr_rbf, ridge]

#对模型进行网格搜索调参
params = {
    'ridge__alpha': [0.001, 0.01, 0.1, 10],
}
scf = StackingRegressor(regressors=models, meta_regressor=ridge)
# 训练回归器
grid = GridSearchCV(estimator=scf, param_grid=params, cv=5, refit=True)
grid.fit(x_train, y_train)
print(grid.best_params_, grid.best_score_)

{'ridge__alpha': 0.001} 0.6916446766499572
```

数据与模型准备

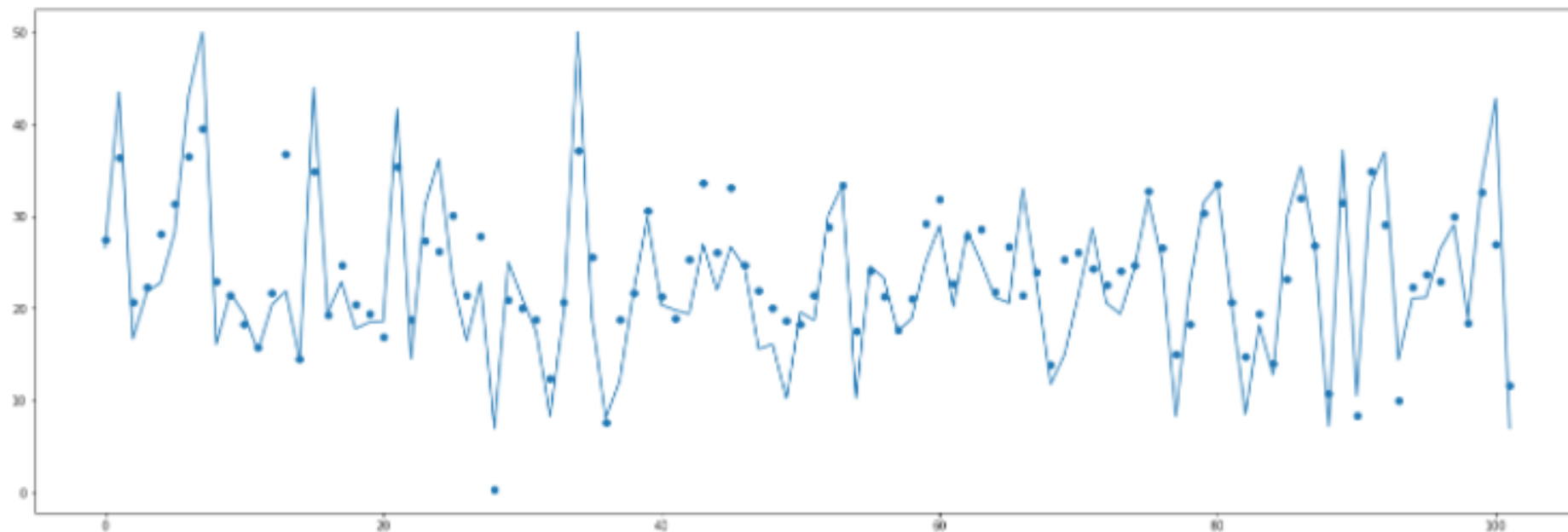
```
#返回到基模型调整参数  
lr = LinearRegression()  
svr_lin = SVR(kernel='linear', gamma='auto')  
svr_rbf = SVR(kernel='rbf', gamma='auto')  
ridge = Ridge(alpha=0.001, random_state=2021)  
models = [lr, svr_lin, svr_rbf, ridge]
```

训练模型与图像绘制

```
# 训练回归器
scf.fit(x_train, y_train)
pred = scf.predict(x_test)

# 图像绘制
print('stacking model')
print("loss is {}".format(mean_squared_error(y_test, pred)))
plt.figure(figsize=(24, 8))
plt.scatter(np.arange(len(pred)), pred)
plt.plot(np.arange(len(y_test)), y_test)
plt.show()
```

```
stacking model
loss is 25.112072393267386
```





Q&A

TRANSWARP
星环科技